

LECTURE 3 · COURSE: NLP, LLMS & AI APPLICATIONS

AI Agents

From Language Models to Autonomous, Tool-Using Systems

 2-Hour Session

 10 Modules

 Hands-on LangGraph Lab

What We'll Cover Today

01

Motivation

10 min

02

What Is an AI Agent?

15 min

03

Agent Components

40 min

04

Workflows vs Agents

20 min

05

Single vs Multi-Agent

15 min

06

Case Studies

30 min

07

When to Use Agents?

15 min

08

LangGraph Mapping

5 min

09

Build an Agent Live

10 min

10

Future of Agentic AI

5 min

Why LLMs Are Not Enough

No Memory Across Turns

Each call starts fresh — there is no persistent state or history unless engineered externally through explicit memory management.

No Access to Real World

Can't check today's stock price, query an internal database, or read a local file without an integration bridge.

No Goal-Directed Planning

Generates the next best token sequentially — it does not decompose complex goals or pursue multi-step objectives autonomously.

No Self-Correction Loop

Produces one answer in a single shot; it will not verify, retry, or fix its own mistakes on its own without external loops.

Evolution: Chatbot → Copilot → Agent

STAGE 1

Chatbot

Answers questions. No action taken.

Example: Early customer-support bots, ChatGPT

STAGE 2

Copilot

Suggests actions. Human executes them.

Example: GitHub Copilot, Notion AI, Email Drafts

STAGE 3

Agent

Plans, acts, and verifies autonomously.

Example: Claude Code, Cursor Agent, AutoGPT

 **Discuss:** Where does your favorite AI tool sit on this spectrum — and what would it take to move it one step to the right?

Formal Definition & Characteristics

"An **AI Agent** is a system that uses an LLM as its reasoning engine to autonomously perceive a goal, plan a sequence of actions, use tools to act on its environment, and adjust its plan based on the results — with minimal human intervention."



Goal-directed

Works toward an objective, not just a single prompt-response.



Autonomous

Chooses its own next action within set boundaries.



Tool-using

Calls external functions, APIs, and data sources.



Adaptive

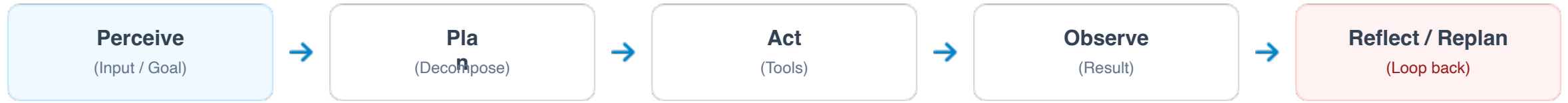
Observes results and revises its plan dynamically.

LLM Alone vs. AI Agent

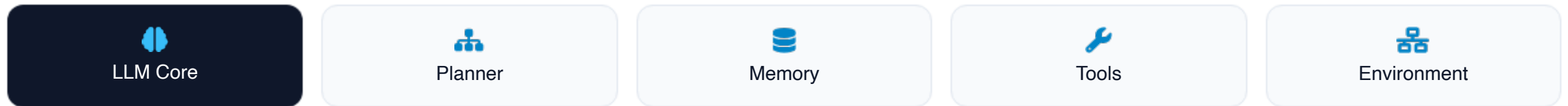
Dimension	LLM Alone	AI Agent
Input → Output	Single prompt → single response	Goal → sequence of planned actions
Memory	None (stateless per call)	Short-term + long-term memory
Tools	None natively	Search, code exec, APIs, databases...
Decision-making	Predicts next tokens	Plans, decides, and revises steps
Error handling	No self-check	Reflection, retries, verification
Autonomy	Fully human-driven	Operates with bounded independence

Agent Lifecycle & Architecture Stack

Agent Lifecycle Loop



Architecture Stack



MODULE 3 · 40 MIN

Agent Components

Deep dive into the 4 core pillars that transform a passive model into an autonomous agent system.

3.1 Planner

Task decomposition & sequence logic

3.2 Memory

Short-term, long-term & state stores

3.3 Tools

APIs, execution & MCP protocol

3.4 Reflection

Self-correction & human-in-the-loop

Turning Goals Into Steps

- ✓ **Goal decomposition:** Breaks a broad objective into ordered sub-tasks.
- ✓ **Task planning:** Sequences sub-tasks and assigns specific tools to each.
- ✓ **Replanning:** Revises the plan when a step fails or context changes.
- ✓ **Reflection:** The planner reviews outcomes before moving on.

LangGraph mapping: Planner → a Node that calls the LLM and returns the next action / route.

Hospital Example

Goal: "Diagnose and schedule treatment for chest pain"

1. Check vitals
2. Order blood lab tests
3. Consult diagnosis agent
4. Check drug interactions
5. Book specialist appointment
6. Notify attending doctor

Planner in Practice & Replanning Loop

INDUSTRY EXAMPLE

Claude Code / Cursor Agent

Given *"fix the failing test,"* the planner reads the stack trace, decides which files to inspect, orders edits before re-running tests, and replans if the fix introduces a new failure.

Replanning Loop:

Plan → Act → Check

✗ fail → replan

✓ pass → next step

LangGraph Implementation

Node Function: Planner logic lives inside a Node function (an LLM call that decides the next step).

Conditional Edge: The routing decision (which tool / which next node) is evaluated dynamically.

Looping Edge: Replanning is simply looping back to the Planner node via a directed edge.

What the Agent Remembers

Short-term

Current conversation / task context — cleared after session completes.

Long-term

Persists across sessions — user preferences, past decisions, facts.

Working

Active scratchpad for the task in progress (similar to CPU RAM).

Semantic Memory

General knowledge and factual concepts the agent has acquired.

Episodic Memory

Specific past events / interactions ("what happened last time").

Key Distinction: Short-term & Working memory $\approx$ active context window. Long-term, Semantic & Episodic memory $\approx$ retrieved from external stores (vector DBs) and injected into prompt context.

Memory Architecture & Flow



Hospital Example

Working memory holds today's lab results while reasoning; long-term/episodic memory recalls patient's medication history and last visit's notes from EHR — even if from months ago.

LangGraph Mapping

Memory = State object threaded through every node.

Short-term memory = Fields updated within a single execution run.

Long-term memory = External checkpointer / vector DB read into State at initialization.

How Agents Act on the World

SEARCH

CALCULATOR

SQL QUERY

PYTHON EXEC

FILE SYSTEM

EXTERNAL APIS

MCP TOOLS

LLM Decides Tool Needed



Formats Tool Call (JSON)



External Tool Executes



Result Returned to LLM

MCP (Model Context Protocol)

Standardizes tool interaction: instead of writing custom glue code per API, tools are exposed through a common protocol the agent can discover and call dynamically — like a **USB port for AI tools**.

Tools in Practice & Design Principles

Hospital Tools Suite

- **Lab API:** Pulls latest bloodwork
- **SQL Tool:** Queries EHR for history
- **Calculator:** Computes dosage by weight
- **Scheduling API:** Books specialist

LangGraph Implementation

- Tools wrapped as **ToolNode** instances.
- LLM emits tool call → Conditional Edge routes to ToolNode.
- Tool output written back into State for next node.

Design Principle: Give the agent the smallest, most precise tool for the job. A narrow `get_patient_labs(id)` beats a broad `run_any_sql_query` — smaller attack surface, safer, easier to execute correctly.

Catching Its Own Mistakes



Self-correction

The agent critiques its own output and revises text/code before finalizing response.



Retry loops

Failed steps are automatically retried, often using an adjusted approach or prompt.



Verification

Output is checked against static validation rules, unit tests, or schema constraints.



Human approval

Human reviews and explicitly confirms high-stakes actions before execution.

The Reflection Loop Mechanics



LangGraph Mapping for Reflection

- **Reflection Loop:** Represented as a directed Edge routing back to an earlier node.
- **Quality Check:** A Conditional Edge evaluating critique criteria to decide exit or loop.
- **Human Approval:** Implemented using `interrupt()` to pause graph execution pending user input.

Where Most Students Struggle

STATIC PATH

Workflow

Follows a fixed, predetermined path every single run.

Invoice → OCR → Extract → Store

DYNAMIC PATH

Agent

Decides its own path dynamically based on context.

Goal → Planner → Dynamic Steps → Tools → Reflection → Done

Key Insight: A workflow cannot handle a step it wasn't explicitly programmed for. An agent can — it replans, selects a different tool, or asks for human help.

Five Terms You Need to Distinguish

Static Workflow

Fixed step sequence, always identical order. No branching based on content.

Dynamic Workflow

Branches based on predefined rules or intermediate step results.

DAG

Directed Acyclic Graph — steps as nodes, dependencies as edges, no loops.

State Machine

Explicit states + transitions triggered by defined events.

Agentic Workflow

LLM runtime decides steps & order — graph discovered on the fly.

Multi-Agent: Hospital System Example

Manager Agent (Orchestrator)

Diagnosis Agent

Drug Agent

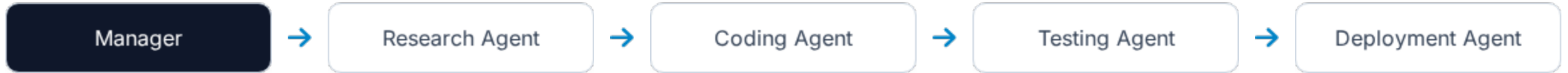
Appointment Agent

Billing Agent

Compliance Agent

Why split it up? Each sub-agent is a specialist with its own tools, prompts, and guardrails (e.g. Compliance Agent enforces HIPAA rules independently). Keeping agent scopes narrow improves reliability.

Software Pipeline & Decision Guide



Single Agent Works When...

The task is narrow enough for a single context window and unified tool set — e.g. "fix this single function."

Multi-Agent Works When...

Distinct phases require specialized expertise, prompt rules, or separate tool sets — research vs coding vs deployment.

Hospital AI — End-to-End Trace

1. Doctor inputs goal: diagnose patient

2. Planner builds execution graph

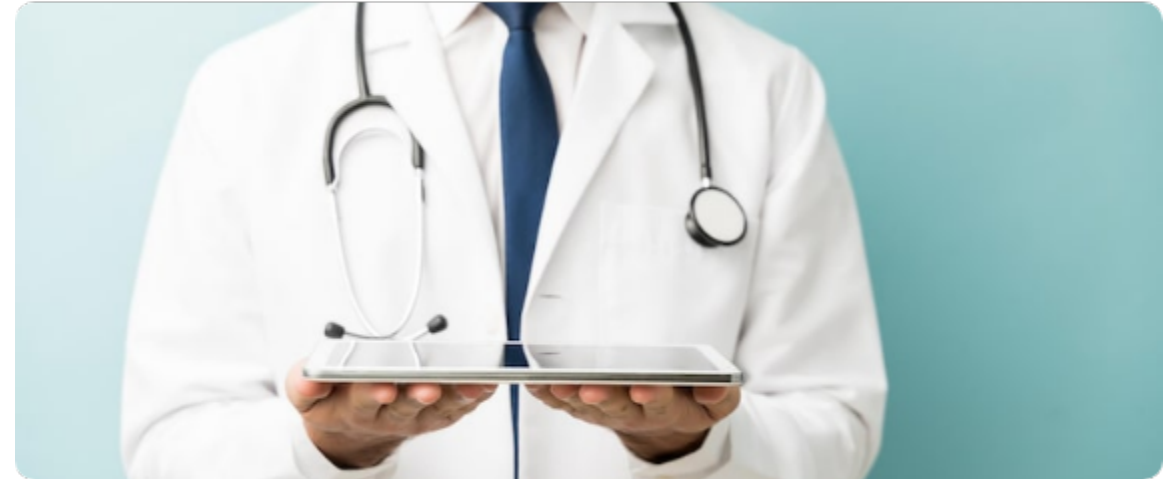
3. EHR Tool pulls patient history

4. RAG retrieves medical literature

5. Lab API fetches results

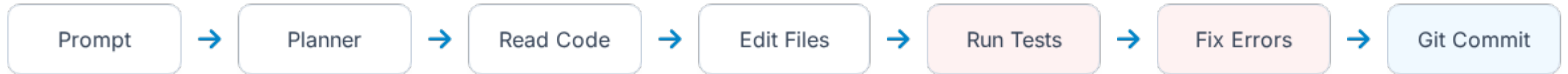
6. Scheduling books specialist

7. Reflection verifies consistency



Teaching Note: This trace touches every core component from Module 3 (Planner, Memory, Tools, Reflection) in a single real-world healthcare interaction.

Coding Agent (Cursor / Claude Code)



Autonomous Reflection Loop

The **"Run Tests"** and **"Fix Errors"** steps form an autonomous loop that repeats until the entire unit test suite passes cleanly, ensuring code quality before committing.

Financial Analyst Agent

User Goal: "Analyze last quarter's sales. Generate charts. Create a PPT. Email the CEO."

Read Excel



Python Analysis



Generate Charts



Create PPT



Email CEO

Why This Is a Great Teaching Example

It chains multiple distinct tool types (file I/O, code execution, document generation, communication APIs) around a single goal — with each tool's output passing directly to the next step.

Practical Decision Tree

Decision Tree Logic

Need multiple steps?

- └─ No → *Use Standard LLM*
- └─ Yes ↓

Need tools / external data?

- └─ No → *Use Standard LLM*
- └─ Yes ↓

Need dynamic planning / adaptation?

- └─ No → *Use Static Workflow*
- └─ Yes → **USE AN AI AGENT**

Why This Matters in Industry

- Agents increase token cost, latency, and surface potential failure modes.
- If a fixed pipeline solves the problem reliably, ship the workflow — it's cheaper and easier to debug.
- Reserve agents for tasks where execution steps genuinely cannot be known in advance.

Concepts → LangGraph Reference

Agent Concept	LangGraph Primitive
Planner	Node (LLM call deciding step)
Memory	State (Shared state object)
Tool	ToolNode
Decision	Conditional Edge
Reflection	Loop (Edge to earlier node)
Workflow	Graph
Final Output	END Node

Live Build Graph & Future Trends

MODULE 9

Build an Agent Live

User Query

Planner Node

Search Tool

Calculator

LLM Response → Answer

MODULE 10

Future of Agentic AI

- **AI Operating Systems:** MCP & A2A protocols
- **Frameworks:** LangGraph, OpenAI Agents SDK, CrewAI
- **Multi-agent Orchestration:** Distributed agent teams
- **Observability & Human-in-the-Loop**

HANDS-ON · 30 MIN

Lab: Build a LangGraph Agent

1. Planner Node

Decides tool required

2. Search & Calc Tools

Wired as ToolNodes

3. Memory State

Shared across execution

4. Reflection Loop

Conditional edge checks answer

5. Final Answer

Routes output to END node

Let me know your questions!

Thank You!

Questions & Discussion — Lecture 3 Complete

Image Sources



https://img.magnific.com/free-photo/midsection-male-practitioner-holding-digital-tablet-while-standing-against-blue-background_662251-356.jpg?semt=ais_hybrid&w=740&q=80

Source: www.magnific.com